

Project-Ticket

A source-code and configuration audit of the Fute Services ticket/HR/IT management platform — React SPA over an Express + Firebase Admin API, deployed as two Vercel projects. Read-only: nothing in the codebase was changed to produce this report.

Scope D:\Project-ticket\Project-Ticket **Date** 2026-08-29

Method Static source review, no live/destructive testing

Repo github.com/Fute-Services/Project-Ticket @ main

78	64	74	72	70	61
ARCHITECTURE	SECURITY	CODE QUALITY	PERFORMANCE	MAINTAINABILITY	PROD READINESS

01 Executive Summary

Project-Ticket is a role-based internal operations platform (tickets/complaints, HR desk, leave, assets, approvals, coordinator tasks, production renders) built as a React 18 + Vite SPA talking to an Express API, which in turn is a thin, consistent layer over Firebase (Firestore for data, Firebase Auth for credentials, Firebase Storage for documents). It is hosted as two independent Vercel projects — `project-ticket` (frontend) and backend (API) — under one Vercel org, with no Docker and no CI/CD pipeline found in the repository.

The codebase shows evidence of deliberate, prior security hardening rather than a first pass: server-side ownership checks on every mutating complaint route, a session-revocation layer that closes the "long-lived JWT can't be revoked" gap, brute-force logout with IP logging, an allow-listed CORS policy, HTML-escaping in outbound email, and a Multer upload pipeline with MIME/size limits. `docs/` already contains a prior `PRODUCTION_READINESS_AUDIT.md` (2026-08-24, 24 findings) and a `SECURITY_REMEDIATION_REPORT.pdf` — this report should be read as a fresh independent pass, cross-checked against those rather than assumed to supersede them.

The material gaps are consistent and structural rather than scattered: no helmet (zero security response headers anywhere), rate limiting confined to the four auth endpoints (every other write/read route is unthrottled), no minimum password policy enforced by the app itself, and two routes (`renderRoutes.js`, the task-status PATCH in `coordinatorRoutes.js`) with no role restriction at all — by design per inline comments, but worth a second look. A sensitive-sounding `docs/login-credentials.pdf` sits in the repo working tree (git-ignored, not tracked — confirmed not committed) and should be removed or relocated regardless.

Bottom line — this is a mid-maturity internal tool with real, working access-control logic on the backend (not merely hidden in the UI), let down by missing perimeter hardening (headers, general rate limiting) that is inexpensive to add. Nothing found in this pass rises to CRITICAL; three findings are rated HIGH. See §24 for the full register.

02 Project Overview

Project-Ticket serves seven role types — founder, superadmin, hr, it, coordinator, employee, plus five demo-only department roles with no backend wiring (App.jsx:54-56) — through one shared dashboard shell. Core domains: HR and IT complaint/ticket queues (shared implementation via a controller factory), an approvals workflow, leave requests, IT asset tracking, an HR "desk" module (employees, candidates, interviews, attendance, job postings), coordinator tasks/projects, and a production "renders" tracker.

```
Project-Ticket/ ├── .vercel/ - root Vercel project link ("project-ticket") ├──
assets/design-tokens.json - design tokens → generates frontend CSS ├── docs/ - 13
markdown + 5 PDF docs; PRD/TRD/prior audits kept current ├── scripts/tokens-to-
css.cjs - build helper, tokens.json → tokens.css ├── .tools/jdk-21.../ - bundled
JDK, unreferenced by any package.json (stray) ├── package-lock.json (root) -
orphaned: no matching root package.json ├── README.md - 17 bytes, placeholder only
└── main/ ├── backend/ - Express API (own Vercel project + Firebase project) | ├──
config/ controllers/ middleware/ routes/ utils/ └── frontend/ - Vite + React 18 SPA
└── src/{components,pages,context,hooks,lib,data,styles,utils}/
```

03 Technology Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE	NOTES
Frontend	React + Vite	React 18.2, Vite 5	SPA, role-based dashboards	Mixed .jsx/.ts, no strict TS enforcement
Routing	react-router-dom	6.21	Client routing, lazy-loaded routes	—
UI Kit	shadcn/ui + Radix + Tailwind	—	Component primitives & styling	—
HTTP client	axios	1.6	API calls, single instance	Interceptors for auth header + 401 handling
Backend	Express	^4.18.2	REST API	No helmet, no morgan
Auth	Firebase Auth + jsonwebtoken	firebase-admin ^12.0.0, jsonwebtoken ^9.0.2	Credential store + app-issued JWT session	Hashing delegated to Firebase (scrypt)
Database	Firestore	via firebase-admin ^12.0.0	Primary data store, schemaless	Admin SDK only — no client Firestore usage found
File storage	Firebase Storage	via firebase-admin	HR-desk employee documents	Multer memoryStorage → bucket, MIME allow-list
Email	Nodemailer (SMTP)	^6.9.7	Complaint/notification emails	HTML-escaped interpolation
Hosting	Vercel	—	Two projects: frontend + backend	No vercel.json found for backend
CI/CD	— none found —	—	—	No .github/workflows, no other CI config
Testing	Playwright	e2e	Frontend e2e tests	tsc --noEmit for typecheck

04 Complete Project Structure

Every folder's role, condensed from the five-agent inventory. **CONFIRMED** unless marked.

main/backend

- `server.js` — entry point, middleware stack, error handler, graceful shutdown
- `config/firebase.js` — Admin SDK init, emulator fallback when creds absent
- `controllers/` — one file per domain; `complaintControllerFactory.js` generates both HR and IT queues from shared logic
- `routes/` — 10 route files, mounted in `server.js`
- `middleware/` — `authMiddleware` (JWT+session check), `roleMiddleware` (allow-list), `permissionMiddleware` (Firestore-backed fine-grained matrix, IT assets only)
- `utils/` — `sessions.js`, `auditLog.js`, `upload.js` (Multer), `mailer.js`, `pagination.js`

main/frontend/src

- `App.jsx` — router, all context providers, lazy route map, `RequireAuth` gating
- `context/` — 9 domain contexts (Auth, Leave, Ticket, Approval, Permissions, TaskProject, Render, Asset, HrDesk)
- `utils/api.js` — single axios instance + ~60 endpoint wrapper functions
- `components/RequireAuth.jsx` — route guard: auth check, role allow-list, skeleton during restore
- `pages/` — one dashboard per role, lazy-loaded
- `tests/e2e/` — Playwright specs

Root-level oddities **flag, not urgent**

- `package-lock.json` at repo root with no accompanying `package.json` — **UNKNOWN** why it exists; likely leftover from a prior root-level setup.

- `.tools/jdk-21.0.12+8/` — a full bundled Windows JDK, referenced by nothing in either `package.json`. UNKNOWN purpose; safe candidate for removal if confirmed unused.
 - `docs/login-credentials.pdf` — filename implies stored credentials; CONFIRMED not tracked by git (gitignore pattern matches), but its presence in a shared docs folder on disk is itself a risk regardless of VCS status.
-

05 Architecture

In plain terms: the browser only ever talks to the Express API; the API is the sole holder of Firebase Admin credentials and the only thing that ever touches Firestore, Firebase Auth, or Firebase Storage.

```
Browser (React SPA, Vercel: project-ticket)
|   HTTPS + Bearer JWT (Authorization header)
▼
Express API (Vercel: backend)
|
|   └─ cors() → allow-list origin check
|   └─ express.json()
|   └─ authMiddleware → verify JWT, check session revoked, check account active
|   └─ roleMiddleware / permissionMiddleware → route-level + resource-level gate
|   └─ controllers/ → business logic, per-request ownership checks
|       |
|       |   └─► Firestore (firebase-admin)      - all app data
|       |   └─► Firebase Auth (Admin SDK + REST) - credential verification
|       |   └─► Firebase Storage                - HR-desk documents
|       |   └─► SMTP (Nodemailer)               - notification email
|       |
|   └─ centralized error handler → generic message to client, full detail server-log
```

06 Frontend Architecture

State is plain React Context per domain (no Redux/Zustand), all mounted in `App.jsx:89-97`. A ponytail: comment at `App.jsx:78-86` already documents a known, accepted inefficiency — every context fetches on login regardless of which page the user actually visits — flagged as low-priority by the team itself. Routing is fully lazy (`React.lazy`) except the login page, with a hand-tuned Rollup `manualChunks` config that groups icon imports and core router/react into shared vendor chunks to avoid 40+ micro-requests — a deliberate, working optimization.

Component → API trace (representative)

COMPONENT	HOOK / CONTEXT	SERVICE CALL	ENDPOINT
LoginPage.jsx	AuthContext	<code>api.login()</code>	POST <code>/api/auth/login</code>
Hr/It ticket views	TicketContext	<code>getComplaints()</code> / <code>createComplaint()</code>	<code>/api/hr it/complaints</code>
Leave pages	LeaveContext	<code>getLeave()</code> / <code>requestLeave()</code>	<code>/api/leave</code>
Approvals views	ApprovalContext	<code>getApprovals()</code> / <code>decide()</code>	<code>/api/approvals</code>
HR Desk module	HrDeskContext	~15 endpoint wrappers	<code>/api/hr-desk/*</code>

Auth on the client

JWT is stored in `localStorage` or `sessionStorage` (key `fute_token`), chosen by a "remember me" toggle — `AuthContext.jsx:38-53,104-129`. An axios request interceptor attaches `Authorization: Bearer <token>` (`api.js:9-13`); a response interceptor force-logs-out on any 401 and hard-redirects to `/login` (`api.js:21-31`). `AuthContext` polls `getMe()` every 15s so role/permission changes made by a superadmin apply without the affected user re-logging in.

No `dangerouslySetInnerHTML`, `innerHTML`, or `eval()` found anywhere under `src/` — React's default JSX escaping is the operative XSS control, and `grep` found no sink that would bypass it. No client-side schema validation library (no `zod/yup/react-hook-form`);

form validation is native HTML5 attributes only, with the server as the real enforcement point.

07 Backend Architecture

Boot sequence in `server.js`: `load env` → `express-async-errors` patches Express 4 to forward rejected promises to the error handler → hard-fail if `JWT_SECRET` is unset (`server.js:16-19`) → `Firebase init` → `cors()` allow-list → `express.json()` → 10 route mounts → health checks → centralized error handler. Graceful shutdown on `SIGTERM/SIGINT` (10s hard-exit fallback), skipped when `process.env.VERCEL` is set since the platform is serverless there.

Request lifecycle

Request

- `authMiddleware` JWT verify, 60s-TTL profile cache, session-revoked check, act:
- `roleMiddleware` route-level allow-list (role in [...])
- `permissionMiddleware` resource-level check (IT assets only, Firestore-backed matrix)
- `controller` validation, ownership check, business logic
- `Firestore` (transaction where state must stay consistent, e.g. status+approval writes)
- `response` success payload, or generic error via central handler

Password hashing is intentionally absent from this codebase — delegated entirely to `Firebase Auth` (no `bcrypt/argon2` dependency, correct given the architecture). The complaint system (HR + IT queues) is generated from one shared factory, `controllers/complaintControllerFactory.js`, rather than duplicated per department — good reuse discipline, and the place where ownership/role checks for complaint mutation live (`lines 262-317`).

08 Database Architecture

Technology — **CONFIRMED** Firestore only, via `admin.firestore()` in `config/firebase.js:44-51`. `@supabase` and `@google-cloud` packages visible in `node_modules` are transitive dependencies of `firebase-admin/google-auth-library`, not directly used — confirmed against `package.json` dependency list, and `docs/TRD.md` independently notes the project moved off an earlier Supabase/Postgres draft to Firebase.

Collections (inferred from `db.collection(...)` call sites — schemaless, no migration files exist)

COLLECTION	TOUCHED BY
users	authController, superAdminController, departmentController, dashboardController, securityController, authMiddleware
hr_complaints / it_complaints	complaintControllerFactory, dashboardController, analyticsController, slaController
approvals	approvalController, complaintControllerFactory, analyticsController
leave_requests	leaveController, analyticsController
assets, departments, tasks, projects	assetController, departmentController, taskProjectController
settings	systemSettingsController, permissionMiddleware, slaController
sessions, failed_logins, audit_logs	utils/sessions.js, authController, utils/auditLog.js
employees, candidates, attendance, sent_emails	hrDeskController
renders	renderController

No firestore.rules file anywhere in the repo. Because every Firestore access goes through the server-side Admin SDK (which bypasses Security Rules by design), this is architecturally consistent rather than an oversight — but it means *all* access control lives in Express middleware/controllers with zero defense-in-depth at the database layer. If a client-side Firebase SDK is ever added to the frontend, it would inherit no rules at all.

confirmed no client Firebase SDK exists today .

09 API Surface

All routes require authMiddleware unless marked public. **CONFIRMED** from route files.

ROUTE FILE	ENDPOINTS
authRoutes.js	POST /register, /login (public, rate-limited) · GET /me · POST /verify-password (rate-limited), /logout
hrRoutes.js / itRoutes.js	complaints CRUD, status update, field update, search-by-token
approvalRoutes.js	POST/GET /, PATCH /:id/decide
coordinatorRoutes.js	projects, tasks, PATCH /tasks/:id/status
founderRoutes.js	~30 endpoints – users, analytics, audit-logs, SLA, notifications, permissions, settings, departments
hrDeskRoutes.js	employees/candidates/interviews/meetings/attendance/feedback/jobs
leaveRoutes.js	POST /, GET /my · GET /, PATCH /:id/decide
renderRoutes.js	GET/POST/PATCH
securityRoutes.js	sessions, force-logout, failed-logins, locked-accounts, unlock

10 Complete Data Flow

Example — filing an HR complaint:

```
Employee fills complaint form (HrTicketPage)
→ TicketContext.createComplaint() [native HTML5 field checks only]
→ POST /api/hr/complaints (Authorization: Bearer JWT)
→ authMiddleware: verify JWT → check session not revoked → check account active
→ roleMiddleware: any authenticated role may create
→ complaintControllerFactory.create(): required-field check, generates 6-char token,
  writes doc to hr_complaints, links to users doc
→ mailer.js: HTML-escapes fields, sends notification email to HR queue via SMTP
→ response: created complaint (incl. token) → TicketContext state → UI list re-render
```

Example — HR resolves and deletes a complaint: PATCH `/:id/status` (hr/founder only) runs inside a Firestore *transaction* that writes the status change and creates the linked approvals doc together — no partial-failure state possible (

`complaintControllerFactory.js:204-260`). DELETE is owner-only, server-enforced (`lines 291-317`), and transactionally removes the linked approval record too.

11 Authentication

QUESTION	ANSWER	EVIDENCE
Where does login happen?	POST /api/auth/login → Firebase Identity Toolkit REST <code>accounts:signInWithPassword</code> (Admin SDK can't verify passwords directly)	<code>authController.js:94-112</code>
Password hashing	Delegated entirely to Firebase Auth (scrypt); not implemented in this codebase	no <code>bcrypt/argon2</code> dependency
Token type / expiry	App-issued JWT, 7-day expiry, payload <code>{id,email,role,full_name,sid}</code>	<code>authController.js:55-59,123-127</code>
Refresh mechanism	None — long-lived token compensated by session-revocation check on every request	<code>authMiddleware.js</code>
Logout / revocation	Session doc marked <code>revoked:true</code> in Firestore, checked server-side on each request — real revocation, not just client token deletion	<code>authController.js:188-193</code> , <code>authMiddleware.js:52-53</code>
Brute-force protection	5 failed attempts → account locked, requires superadmin unlock; IP logged to <code>failed_logins</code> ; 10 req/15min rate limit on login/register/verify-password	<code>authController.js:10,102-112</code> · <code>authRoutes.js:12-18</code>
Self-registration role	Hardcoded to employee — comment confirms a prior email-pattern role-guessing bug was fixed here	<code>authController.js:40-45</code>
Privileged account creation	Only via superadmin-gated POST <code>/api/founder/users</code> , with an <code>ASSIGNABLE_ROLES</code> allow-list excluding founder/superadmin	<code>superAdminUserController.js:8</code>
Password policy	Not enforced app-side — registration only checks non-empty; Firebase's default 6-char minimum is the real floor. Admin-driven resets enforce 10 chars minimum, inconsistent with the weaker self-registration floor	<code>superAdminUserController.js:268-289</code>
MFA	UNKNOWN — not found in files read	—

12 Authorization

Three layers, all server-side: `roleMiddleware` (route-level allow-list), `permissionMiddleware` (Firestore-backed fine-grained action matrix, currently applied only to IT asset create/edit/delete, superadmin always bypasses, default-allow when unconfigured), and controller-level ownership checks on mutation endpoints — e.g. complaint update requires owner-or-staff, delete requires owner (

`complaintControllerFactory.js:277-281,297-299`).

Two access-control items worth a second look

- `searchByToken` (`complaintControllerFactory.js:191-198`) lets any authenticated user, any role, fetch *any* complaint by its token — not scoped to the caller. Tokens are 6-char random alphanumeric ($\approx 2.2B$ combinations), so not practically guessable, but this is a horizontal-access design choice (tokens double as a shared status-lookup key) rather than a hardened boundary.
LIKELY minor IDOR by design .
- `coordinatorRoutes.js` PATCH `/tasks/:id/status` has no role gate, and it's **UNKNOWN** (not verified this pass) whether `taskProjectController.js` adds an ownership check underneath.

13 Security Controls Currently Implemented

CONTROL	STATUS	WHERE	STRENGTH
Server-enforced ownership/role checks	Implemented	complaintControllerFactory.js, roleMiddleware, permissionMiddleware	Strong — verified server, merely hidden in UI
Session revocation	Implemented	authMiddleware.js + utils/sessions.js	Strong — closes long-JWT lifetime gap within ~60s TTL
Brute-force / logout	Implemented	authController.js, failed_logins collection	Strong — account lockout logging + rate limit controls
CORS	Implemented	server.js:32-46	Strong — explicit allow-list, wildcard
Error message hygiene	Implemented	server.js:78-89	Strong — generic client errors always, full detail server-side
File upload validation	Implemented	utils/upload.js	Strong — memoryStorage allow-list, 10MB cap
Email injection hardening	Implemented	utils/mailer.js:14-21	Good — HTML-escapes interpolated fields
Audit logging (admin actions)	Implemented	utils/auditLog.js	Good — user create/update/delete/password change/reset logged
Secrets in git	Implemented	.gitignore at root and backend; only .env.example tracked	Confirmed clean via git history
Rate limiting	Partial	authRoutes.js only	Weak elsewhere — generic unthrottled
Password policy	Partial	Firebase default only, inconsistent admin-reset floor	Weak
Security response headers	Missing	—	No helmet, no CSP/HTTP Frame-Options anywhere
Structured/centralized logging	Missing	—	console.log/error only; no morgan/winston/pino
CI/CD security gates	Missing	—	No pipeline exists to run checks
MFA	Unknown	—	Not found in files read

14 Security Gaps

- **No helmet** — zero default security headers (CSP, X-Content-Type-Options, X-Frame-Options, HSTS) anywhere in `server.js`. **HIGH**
- **Rate limiting confined to 4 auth endpoints** — every `hr/it/leave/coordinator/render/hr-desk` route is unthrottled, opening the door to scraping or resource-exhaustion abuse by any authenticated account. **MEDIUM**
- **No enforced password minimum at registration** — relies on Firebase Auth's default 6-char floor; inconsistent with the 10-char floor the app itself enforces on admin-driven resets. **MEDIUM**
- `renderRoutes.js` and the coordinator task-status endpoint have no role gate — intentional per comments, but broadens the blast radius of any compromised low-privilege account. **LOW**
- `searchByToken` horizontal-access design (see §12). **LOW**
- `docs/login-credentials.pdf` present on disk (not in git, but a live risk if the docs folder is ever shared). **MEDIUM**
- No CI/CD means no automated dependency-vulnerability scanning, lint, or test gate before deploy. **MEDIUM**

15 Sensitive Data Flow

Credentials

Enters via login form → sent over HTTPS to Firebase Identity Toolkit (never touches this app's own DB) → app never stores a password, only Firebase does → session represented by app JWT stored client-side in local/sessionStorage.

JWT / session token

Issued server-side → held in browser storage (JS-readable, not httpOnly) → sent as Bearer header on every request → invalidated via Firestore sessions doc on logout or forced revoke by a superadmin.

Employee documents (HR Desk)

Uploaded via Multer (memory, MIME-checked) → written to Firebase Storage bucket → readable only through backend-mediated routes gated to hr/founder.

PII in Firestore

Names, emails, roles in users; complaint text/attachments in complaint collections. No field-level encryption found — protection is entirely access-control-based (Admin-SDK-only access, no client Firestore path, no security rules needed as a result).

16 Third-Party Services

SERVICE	PURPOSE	DATA SENT	AUTH	RISK
Firebase Auth	Credential store, identity	email/password at login, user profile	Service-account cert (Admin SDK) + Web API key (REST)	Low — standard managed auth
Firestore	Primary database	all app data	Same service-account cert	Low — no rules needed since Admin-SDK-only
Firebase Storage	HR document uploads	PDF/DOC/JPG files, ≤10MB	Same service-account cert	Low — MIME/size validated
SMTP (Nodemailer)	Notification email	complaint metadata, escaped	SMTP_USER/PASS env vars	Low — output escaped
Vercel	Hosting (frontend + backend)	—	Vercel account / OIDC	—

No SMS, push notification, analytics, payment, or mapping integrations found anywhere in main/backend or main/frontend (grep for stripe/twilio/sendgrid/segment/mixpanel/mapbox: no hits).

17 Environment Variables & Secrets

Names only, sourced from tracked `.env.example` files and `grep` of `process.env.*` usage.
No values were read or printed.

VARIABLE	REQUIRED	USED WHERE	SENSITIVE
JWT_SECRET	Yes — boot fails without it	server.js, authController	Yes
FIREBASE_PROJECT_ID / CLIENT_EMAIL / PRIVATE_KEY	No — falls back to local emulator	config/firebase.js	Yes
FIREBASE_API_KEY	Yes for real login	authController (Identity Toolkit REST)	Yes
FIREBASE_STORAGE_BUCKET	No — defaults to <project>.appspot.com	config/firebase.js	No
FIREBASE_AUTH_EMULATOR_HOST / FIRESTORE_EMULATOR_HOST	Dev only	config/firebase.js	No
SMTP_HOST / PORT / USER / PASS	Yes for email	utils/mailer.js	Yes
FRONTEND_URL	Yes for prod CORS	server.js	No
PORT	No	server.js	No
VITE_API_BASE_URL	Yes (frontend)	main/frontend/src/utils/api.js	No
VERCEL_OIDC_TOKEN	Platform-managed	root .env.local	Platform token, not a service creden

Secret hygiene — **CONFIRMED clean** `git ls-files | grep -i env` shows only the two `.env.example` template files are tracked; all real `.env/.env.local` files are gitignored. Repo-wide `grep` for key-shaped strings (`AIza`, `sk_live`, `BEGIN PRIVATE KEY`, etc.) found only two benign matches: a hardcoded `test` password in a Playwright helper, and a UI placeholder string in a component — no real secrets in source.

18 Error Handling

A single centralized handler (`server.js:78-89`) logs the full error server-side and returns a generic "Internal server error" to the client unless the thrown error carries an explicit `.status` set intentionally by a controller — meaning raw stack traces and Firestore internals never reach the client, in dev or prod alike (no environment branching needed, since it never exposes detail either way). `express-async-errors` ensures rejected promises in async route handlers are forwarded here rather than crashing the process or hanging the request.

19 Logging & Monitoring

Implemented : an application-level `audit_logs` Firestore collection captures privileged admin actions (user create/update/delete, permission change, password reset) via `utils/auditLog.js`. Failed logins are logged with IP to `failed_logins`. **Missing** : no structured request/error logger (morgan/winston/pino) — only ad hoc `console.log/console.error`, which on Vercel lands in ephemeral function logs, not a queryable store. No APM/monitoring/alerting tool integration found.

20 Performance

High impact

Every context fetches on login regardless of visited page (documented, accepted trade-off — `App.jsx:78-86`). General API routes are unthrottled, so cost/latency scale poorly if usage spikes.

Medium impact

15s polling on `getMe()` plus additional visibility-aware polling elsewhere adds recurring request volume as headcount grows — acceptable at current scale, revisit before a large user-count increase.

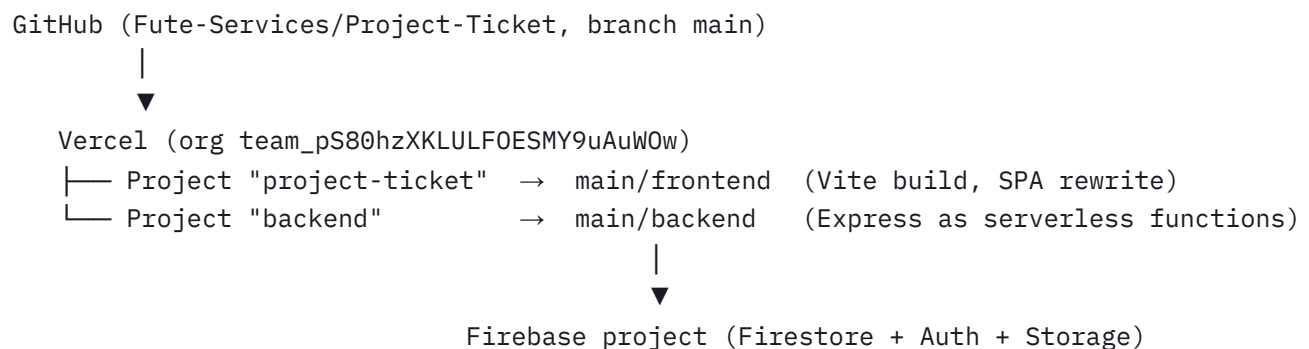
Low impact

Route-level code splitting and a hand-tuned vendor-chunk strategy already address the biggest frontend bundle risk. Only 5 static images in the app — no image-lazy-loading gap of consequence.

21 Code Quality

- **Good reuse discipline:** HR and IT complaint controllers share one factory (`complaintControllerFactory.js`) instead of copy-paste duplication — an explicit comment notes this replaced an earlier duplicated pair.
 - **Self-documenting practice:** `docs/` already contains PRD, TRD, workflow docs, and two prior audit artifacts kept current against the code — unusually mature for a project this size.
 - **Mixed JS/TS:** TypeScript config exists (`tsc --noEmit`) but most files remain `.jsx`, so type-checking coverage is **UNKNOWN** /partial.
 - **Uncommitted work-in-progress** present in the working tree at time of audit (`hrDeskController`, `hrDeskRoutes`, a couple of frontend components, one new untracked component) — normal mid-development state, not a defect.
 - **Orphaned files:** root `package-lock.json` with no matching manifest, and an unreferenced bundled JDK under `.tools/` — low-cost cleanup candidates.
-

22 Deployment Architecture



No CI/CD pipeline and no Docker configuration exist in this repository — no

`.github/workflows`, no `Dockerfile/docker-compose` anywhere in the tree. Deployment is presumably Vercel's native Git-push auto-deploy, but the exact trigger mechanism is **UNKNOWN** from source alone (no `vercel.json` found for the backend project to confirm build settings). TLS termination is handled at Vercel's edge; no app-level HTTPS-redirect middleware was found, consistent with that hosting model.

23 Feature Map

FEATURE	FRONTEND	API	AUTH GATE	DI
HR / IT Complaints	TicketContext, role dashboards	hrRoutes/itRoutes → complaintControllerFactory	role + owner-or-staff on edit/delete	h / it
Approvals	ApprovalContext	approvalRoutes	it/hr/founder create; founder decides	ap
Leave	LeaveContext	leaveRoutes	self-service any; hr/founder decide	le
IT Assets	AssetContext	itRoutes (assets)	role + permissionMiddleware matrix	as
HR Desk	HrDeskContext	hrDeskRoutes	hr/founder (+scoped exceptions)	er ca at se
Coordinator Tasks/Projects	TaskProjectContext	coordinatorRoutes	write: coordinator/founder; status PATCH: none	ta
Production Renders	RenderContext	renderRoutes	none (by design)	re
Admin / Founder console	Founder dashboards	founderRoutes	superadmin (mostly)	u d a p
Security console	Founder security views	securityRoutes	superadmin only	se fa

24 **Security Risk Register**

RISK	SEVERITY	LOCATION	RECOMMENDATION
No security response headers	HIGH	server.js	Add <code>helmet()</code> — one line, no behavior risk
General API routes unthrottled	HIGH	hr/it/leave/coordinator/render/hr-desk routes	Apply a moderate global <code>express-rate-limit</code> in addition to the strict auth-route limiter
docs/login-credentials.pdf present on disk	HIGH	docs/	Remove or relocate outside the shared repo folder regardless of git-ignore status
No enforced password policy at registration	MEDIUM	authController.js register/createUser	Enforce the same 10-char (or stronger) floor used for admin resets
No CI/CD → no automated dependency/lint/test gate	MEDIUM	repo-wide	Add a minimal GitHub Actions workflow running <code>npm audit/typecheck/e2e</code> on PRs
No structured logging/monitoring	MEDIUM	backend	Add a lightweight logger (pino) and ship to a queryable sink
<code>renderRoutes / task-status PATCH</code> have no role gate	LOW	<code>renderRoutes.js</code> , <code>coordinatorRoutes.js</code>	Confirm this is still intentional; add role/ownership check if not
<code>searchByToken</code> cross-user lookup	LOW	<code>complaintControllerFactory.js</code> :191-198	Acceptable given token entropy; document as intentional
Orphaned root <code>package-lock.json</code> / unused JDK bundle	LOW	repo root, <code>.tools/</code>	Delete if confirmed unused — reduces repo confusion, not a security issue

25 Trust Boundary Analysis

User —(1)— Browser —(2)— Internet/TLS —(3)— Vercel edge —(4)— Express API —

(1) Browser holds JWT in local/sessionStorage – JS-readable, no httpOnly cookie.

Mitigated by: no innerHTML/eval sinks found in the SPA's own code.

(2) HTTPS enforced by Vercel's TLS termination; no app-level HSTS/redirect (helmet gap,

(3) CORS allow-list restricts which origins the API accepts requests from.

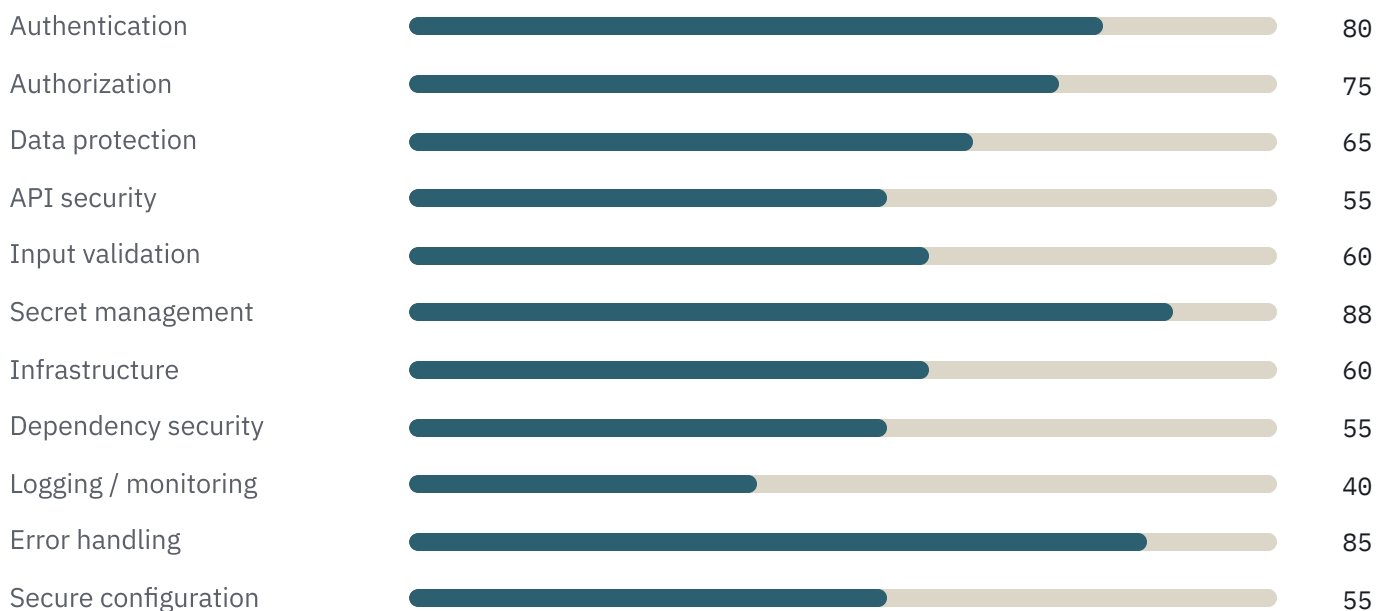
(4) JWT verified + session-revocation + role/permission checks before any Firestore acc

(5) Firebase Admin SDK – full-trust credential, server-only, never exposed to the brows

No Firestore Security Rules exist, which is fine ONLY as long as (4) is the sole ac

26 Security Maturity Score

0–100 per category. Scored against what was verified in source, not assumed.



Composite: 64/100. Dragged down mainly by missing headers (API security), absent structured logging, and no automated dependency scanning (no CI/CD). Authentication, error handling and secret hygiene are the strongest categories – all backed by concrete, working mechanisms rather than partial or assumed controls.

27 Final Project Health Score

Architecture – 78/100

Clean separation of frontend/API/data tiers, consistent controller-factory reuse, sound request lifecycle. Docked for the no-rules-at-the-DB-layer single point of dependence on server-side enforcement.

Security – 64/100

See §26. Solid foundations, missing perimeter hardening.

Code Quality – 74/100

Good factoring and self-documentation habits; docked for mixed JS/TS discipline and a couple of orphaned repo artifacts.

Performance – 72/100

Deliberate code-splitting and chunking already in place; docked for eager context-fetching (self-acknowledged) and lack of general throttling.

Maintainability – 70/100

Strong docs/ practice and shared-factory pattern aid this; multiple controllers not read this pass keep confidence at "good, not fully verified."

Production Readiness – 61/100

Working core flows and access control, but no CI/CD, no security headers, and no monitoring are meaningful gaps before scaling past current usage.

28 Critical Findings

No CRITICAL-severity findings were confirmed in this pass. The three HIGH items are all inexpensive to close: add `helmet()`, extend rate limiting beyond the auth routes, and remove `docs/login-credentials.pdf` from the working tree. None of the three requires an architecture change.

29 Recommended Improvements

1. Add `helmet()` to the Express middleware stack — closes the single largest gap for the least effort.
2. Apply a general-purpose `express-rate-limit` across all routes (looser than the auth-route limiter), not just the four auth endpoints.
3. Enforce a consistent password-length/complexity floor at registration matching the admin-reset standard (10+ chars).
4. Remove `docs/login-credentials.pdf` from the repository working tree; store any real credentials in a password manager instead.
5. Stand up a minimal CI workflow (lint + typecheck + `npm audit` + Playwright smoke) gating merges to main.
6. Add structured logging (e.g. `pino`) shipped to a queryable sink, so failed-request/error visibility doesn't depend on ephemeral Vercel function logs.
7. Revisit whether `renderRoutes.js` and the coordinator task-status endpoint should stay role-open as the user base grows.
8. Clean up the orphaned root `package-lock.json` and the unreferenced `.tools/jdk-21.../bundle` once confirmed unused.
9. Cross-reference this report against the existing `docs/PRODUCTION_READINESS_AUDIT.md` and `SECURITY_REMEDIATION_REPORT.pdf` to reconcile any items already tracked or resolved since 2026-08-24.

30 Final Conclusion

Project-Ticket is a coherently architected internal tool: one API, one data store, consistent server-side enforcement of the access rules that actually matter (ownership, role, session validity), and a codebase that documents its own trade-offs rather than hiding them. The gaps found here are the ordinary, low-drama kind — missing headers, thin rate limiting, a password policy inconsistency, no CI — none of which require redesigning anything, all of which are addressable in a focused week of hardening work rather than a rebuild. Treat §29 as the punch list; nothing in this audit blocks continued use of the application as-is for its current internal audience.

Read-only static audit — no code, configuration, dependencies, or data were modified to produce this report. Every finding above traces to a file path and, where applicable, a line range; items that could not be verified from available source are labeled UNKNOWN rather than assumed.